

Manual Técnico - Plataforma TECHO Chile

Este documento proporciona una visión profunda de la arquitectura, componentes y lógica de negocio de la plataforma. Está dirigido a desarrolladores, administradores de sistemas y personal de TI.

1. Arquitectura del Sistema

La plataforma está construida sobre una arquitectura monolítica modular utilizando el framework **Django (Python)**. Sigue el patrón de diseño **MVT (Modelo-Vista-Template)**, que separa la lógica de datos, la lógica de negocio y la interfaz de usuario.

1.1. Tecnologías Base

- **Backend:** Python 3.13 + Django 5.x
- **Base de Datos:** SQLite (Entorno Desarrollo) / PostgreSQL (Recomendado Producción).
- **Frontend:** HTML5, CSS3 (Variables CSS nativas), Bootstrap 5.3.
- **Interactividad:** JavaScript Vanilla + HTMX (para actualizaciones parciales del DOM sin recarga).
- **Iconografía:** Bootstrap Icons.

1.2. Estructura de Directorios

techo-django/

```
├── config/      # Configuración global (settings, urls, wsgi)
├── core/        # Aplicación principal (lógica de negocio)
│   ├── models.py  # Definición de tablas y relaciones
│   ├── views.py   # Controladores de vistas
│   ├── forms.py   # Formularios y validaciones
│   └── admin.py   # Configuración del admin de Django
├── templates/   # Plantillas HTML
└── layout/      # Base.html y componentes estructurales
```

| └─ reports/ # Plantillas para generación de PDF
| └─ static/ # Archivos CSS, JS e imágenes
| └─ media/ # Archivos subidos por usuarios (evidencias)

2. Modelo de Datos (Core)

El corazón del sistema reside en

core/models.py. A continuación se detallan las entidades principales y sus relaciones.

2.1. Estructura Habitacional

- Proyecto: Entidad padre. Contiene metadatos del proyecto (código, ubicación, fechas).
 - *Relación:* 1 Proyecto tiene N Viviendas.
- Vivienda: Unidad física. Almacena dirección, características (tipo, modelo) y datos del propietario actual.
- FichaInmueble: Tabla intermedia crítica que vincula

Proyecto y

Vivienda con datos técnicos de entrega.

- *Importante:* Es la base para el cálculo de plazos DS 49.

2.2. Gestión de Postventa

- RegistroPostventa (**Ticket**): Representa un incidente reportado.
 - *Campos Clave:* urgencia,

estado (flujo de trabajo), estado_seguimiento (aprobación).

- *Relación:* Vinculado a

FichaInmueble (dónde ocurre) y User (quién reporta).

- Evidencia: Archivos adjuntos (fotos/docs) asociados a un Registro.
- RegistroComentario: Bitácora de comunicación asociada a un Registro.

2.3. Usuarios y Roles

- PerfilUsuario: Extensión del modelo User nativo de Django.
 - *Campo rol*: Define los permisos (Admin, Trabajador, Familia).
 - *Relación*: 1 User tiene 1 PerfilUsuario.

3. Lógica de Negocio Crítica

3.1. Cálculo de Plazos DS 49

El sistema implementa una lógica automática para determinar el estado de garantía de una vivienda según el Decreto Supremo 49.

- **Ubicación:**

core/models.py -> FichaInmueble.estado_ds49()

- **Lógica:** Se calcula la diferencia en días entre fecha_entrega y timezone.now().date().

- **Estados Calculados:**

- NORMAL: ≤ 30 días.
- ADVERTENCIA: 31-60 días.
- CRITICO: 61-120 días.
- VENCIDO: > 120 días.

3.2. Sistema de Notificaciones

No utiliza websockets, sino un modelo de base de datos (

Notificacion) consultado en cada carga de página (context processor).

- **Disparadores:** Creación de comentarios, cambio de estado de ticket, asignación de tareas.
- **Visualización:** Inyectado en

base.html a través de

context_processors.py (o lógica en vista), mostrando un contador en la campana del navbar.

4. Guía de Despliegue y Mantenimiento

4.1. Requisitos del Servidor

- Sistema Operativo: Linux (Ubuntu/Debian recomendado) o Windows Server.
- Python 3.10 o superior.
- Servidor Web: Gunicorn (Linux) o Waitress (Windows) detrás de Nginx/Apache.

4.2. Variables de Entorno

Configure un archivo .env en la raíz con:

- SECRET_KEY: Llave criptográfica única.
- DEBUG: False en producción.
- ALLOWED_HOSTS: Dominios permitidos.
- DATABASE_URL: Cadena de conexión a BD.

4.3. Comandos de Mantenimiento

- **Aplicar cambios en BD:** python manage.py migrate
- **Recopilar estáticos:** python manage.py collectstatic
- **Crear superusuario:** python manage.py createsuperuser

4.4. Solución de Problemas Comunes

- **Error "TemplateSyntaxError":** Generalmente tags de Django mal cerrados en HTML. Revisar logs para línea exacta.
- **Archivos no cargan (404):** Verificar configuración de STATIC_ROOT y MEDIA_ROOT en

settings.py.

- **Correos no llegan:** Verificar credenciales SMTP en

settings.py.

5. Extensibilidad

Para agregar nuevas funcionalidades:

1. Defina el modelo en

`models.py`.

2. Cree la migración: `makemigrations` y `migrate`.

3. Cree la vista en

`views.py` y la URL en

`urls.py`.

4. Cree el template HTML en `templates/`.

5. Si requiere permisos, use el decorador `@user_passes_test` o verifique `request.user.perfilusuario.rol`.